

Financial Signal Processing in Python IX - Wavelet Data Denoising

Part IX of Decomposing Time Series and Understanding their Components



SOFIEN KAABAR, CFA

AUG 29, 2025



7



Sh



This article will be the part IX of a series of articles that present the signal processing field in an easy and straightforward manner. **Financial Signal Processing** (FSP) is the application of signal processing techniques to financial time series—like stock prices.

Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our [privacy policy](#).

Manage

Reject

Accept

forecasts.

There are many concepts in FSP that are worth discussing, and throughout these special articles, I will try to present each one with functioning code that shows how use and interpret it:

- **Decomposition**

- Empirical
- Principal C

- **Filters**

- Moving Av
- Kalman Fil
- Wiener Fil

- **Spectral Analy**

- Fourier Tr
- Wavelet Tr

- **Denoising**

- Wavelet Denoising 📌
- Savitzky-Golay Filter

- **Anomaly Detection**

- Recurrence Plots
- Entropy
- Signal Snikes



Discover more from Engineering Alpha
Sharing in-depth machine learning, quantitative, and technical market knowledge.
Over 4,000 subscribers

Subscribe

By subscribing, you agree Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).

Already have an account? [Sign in](#)

Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our [privacy policy](#).

Many time series can be thought of as a clean signal $\mathbf{S}t$ observed in noise:

$$x_t = s_t + \varepsilon_t, \quad t = 1, \dots, n,$$

with εt often modeled as *i.i.d.* Gaussian noise. A good denoiser should suppress εt without erasing salient structure in $\mathbf{S}t$ (levels, kinks, bursts, periodicities). Wavelets effective because they give a **sparse, multi-scale** description of piecewise-smooth signals. Apply an orthonormal discrete wavelet transform (DWT) $\mathbf{W}$ to $\mathbf{x}$:

$$\mathbf{w} = \mathbf{W}\mathbf{x} = (a_J, d_J, d_{J-1}, \dots, d_1),$$

where a_J are coarse (approximation) coefficients and d_j are detail coefficients at scale $j=1$ (finest) to J (coarsest). Because $\mathbf{W}$ is orthonormal, energy is preserved (Parseval):

$$\|\mathbf{x}\|_2^2 = \|\mathbf{w}\|_2^2.$$

White noise in time remains white noise in wavelet space, sprinkling small coefficients across all scales, while meaningful structure concentrates in relatively **large** coefficients. This separation motivates **shrinkage**: keep big coefficients (likely signal), suppress small ones (likely noise).

The standard recipe is **soft thresholding** of detail coefficients, leaving the approximation untouched to preserve large-scale trends:

$$\tilde{d}_{j,k} = \text{sign}(d_{j,k}) (|d_{j,k}| - \lambda_j)_+, \quad \tilde{a}_J = a_J,$$

Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our [privacy policy](#).

A widely used choice is the (level-wise) **universal threshold**:

$$\lambda_j = \hat{\sigma}_j \sqrt{2 \log n},$$

where σ is a robust noise estimate at scale j , typically from the median absolute deviation (MAD) of the finest-scale (or per-scale) detail coefficients:

$$\hat{\sigma}_j = \frac{\text{median}(|d_{j,\cdot}|)}{0.6745}.$$

This controls the probability of retaining pure-noise coefficients. Alternatives include SURE, BayesShrink, and cross-validated thresholds, but the universal rule is simple, fast, and reliable when noise is close to white.

What about boundary effects and shift sensitivity?

Finite signals require extension at the edges (e.g., symmetric padding). Also, the critically sampled DWT is **not translation-invariant**: small circular shifts of $\mathbf{x}$ can change which coefficients cross λ . A pragmatic fix is **cycle-spinning**: denoise multiple circularly shifted versions and average them:

$$\tilde{\mathbf{x}} = \frac{1}{S} \sum_{s=0}^{S-1} \text{roll}_{-s} \left(\mathcal{D}(\text{roll}_s(\mathbf{x})) \right),$$

where $\mathcal{D}(\cdot)$ denotes wavelet-threshold denoising. This approximates the translation-invariant (stationary) wavelet transform at low cost and visibly reduces “ringing” and

Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our [privacy policy](#).

engineering (transients). For a random walk $S_t = S_{t-1} + \eta_t$ itself, the path is intrinsic rough; however, if the *observations* are noisy, i.e.,

$$y_t = S_t + \varepsilon_t,$$

Wavelet denoising still helps by attenuating the *measurement* noise ε_t while keeping the stochastic trend.

*Do you want to master **Deep Learning** techniques tailored for time series, trading, and market analysis?*

*My book breaks it all down from basic **Machine Learning** to complex multi-period **LSTM forecasting** while going through concepts such as **Fractional Differentiation** and **Forecasting Thresholds**. Get your copy here !*

Application of the Machine Learning Model

The code creates a noisy random walk (a path that moves up and down unpredictably with added measurement noise), then tries to clean it up using wavelet denoising.

Wavelets break the signal into big-picture trends and fine details; the fine details mostly contain noise, so we shrink or remove them and then rebuild the signal. The result is a smoother version of the noisy path that keeps the main movements of the random walk while reducing the extra fuzz.

Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our [privacy policy](#).

```
def wavelet_denoise(x, wavelet="db8", level=None, mode="symmetric",
                   spins=0, per_level=True, keep_approx=True):
    """
    Wavelet soft-threshold denoising with optional cycle-spinning.

    Parameters
    -----
    x : array-like
        1D signal.
    wavelet : str
        PyWavelets wavelet name (e.g., 'db8', 'sym8', 'coif5').
    level : int or None
        Decomposition level. If None, uses pywt.dwt_max_level(len(x), filter_len).
    mode : str
        Signal extension mode: 'symmetric', 'periodization', etc.
    spins : int
        Number of cycle-spins (0 = none). If >0, uses shifts s=0..spins-1.
    per_level : bool
        If True, estimate sigma and threshold at each level separately.
        If False, estimate once from the finest level and use same lambda for all
        details.
    keep_approx : bool
        If True, do not threshold the approximation coefficients.

    Returns
    -----
    x_hat : np.ndarray
        Denoised signal, same length as x.
    thresholds : list
        Thresholds used at each detail level (from 1 down to 1)
```

Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our [privacy policy](#).

```

coeffs = pywt.wavedec(sig, wavelet=wavelet, mode=mode, level=level)
aJ, d_details = coeffs[0], coeffs[1:]
n = sig.size

# Level-dependent sigma and thresholds
thresholds = []
new_coeffs = [aJ.copy()]

# detail order in coeffs is [dJ, dJ-1, ..., d1]
for d in d_details:
    if per_level:
        sigma = np.median(np.abs(d)) / 0.6745 if d.size > 0 else 0.0
    else:
        # if using a single sigma from the finest scale, compute once
        pass
    thresholds.append(sigma * np.sqrt(2.0 * np.log(n)))

if not per_level:
    # compute sigma from the finest scale d1 (last in list)
    finest = d_details[-1]
    sigma_global = np.median(np.abs(finest)) / 0.6745 if finest.size > 0 else
0.0

    lam = sigma_global * np.sqrt(2.0 * np.log(n))
    thresholds = [lam for _ in d_details]

# Soft-threshold details
denoised_details = []
for d, lam in zip(d_details, thresholds):
    denoised_details.append(np.sign(d) * np.maximum(np.abs(d) - lam, 0.0))

# Optionally threshold approximation (usually we keep it)

```

Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our [privacy policy](#).

```

        aJ_new = np.sign(aJ) * np.maximum(np.abs(aJ) - lamA, 0.0)

    coeffs_new = [aJ_new] + denoised_details
    # Reconstruct
    x_hat = pywt.waverec(coeffs_new, wavelet=wavelet, mode=mode)
    # Match original length (waverec can overshoot by < filter_len)
    return x_hat[:n], thresholds

if level is None:
    # choose a reasonable maximum level for the signal and wavelet filter length
    maxlev = pywt.dwt_max_level(data_len=x.size,
filter_len=pywt.Wavelet(wavelet).dec_len)
    level = max(1, min(10, maxlev)) # cap to avoid silly levels on long signals

if spins and spins > 0:
    acc = np.zeros_like(x, dtype=float)
    # store one set of thresholds for reporting (first shift)
    thresholds_used = None
    for s in range(spins):
        xs = np.roll(x, s)
        x_hat_s, th_s = _single_denoise(xs)
        if thresholds_used is None:
            thresholds_used = th_s
        acc += np.roll(x_hat_s, -s)
    return acc / float(spins), thresholds_used
else:
    return _single_denoise(x)

# ----- Demo: Random walk + observation noise + denoising -----

```

```

def simulate_random_walk(n=1024, drift=0.0, step_sigma=1.0, obs_sigma=0.5, seed=13):

```

Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our [privacy policy](#).

```

if __name__ == "__main__":
    n = 2048
    S, y = simulate_random_walk(n=n, drift=0.01, step_sigma=1.0, obs_sigma=0.8, seed=7

# Denoise with and without cycle-spinning
y_hat, thr = wavelet_denoise(
    y,
    wavelet="db8",
    level=None,          # auto
    mode="symmetric",
    spins=0,             # no cycle-spinning
    per_level=True,
    keep_approx=True
)

y_hat_cs, thr_cs = wavelet_denoise(
    y,
    wavelet="db8",
    level=None,
    mode="symmetric",
    spins=8,             # 8 shifts for translation robustness
    per_level=True,
    keep_approx=True
)

# Plot
plt.figure(figsize=(11, 6))
plt.plot(S, color="black", lw=1.5, label="Latent random walk $S_t$")
plt.plot(y, color="tab:gray", alpha=0.45, lw=1, label="Noisy observations $y_t$")
plt.plot(y_hat, color="tab:blue", lw=1.5, label="Wavelet denoised (no spins)")

```

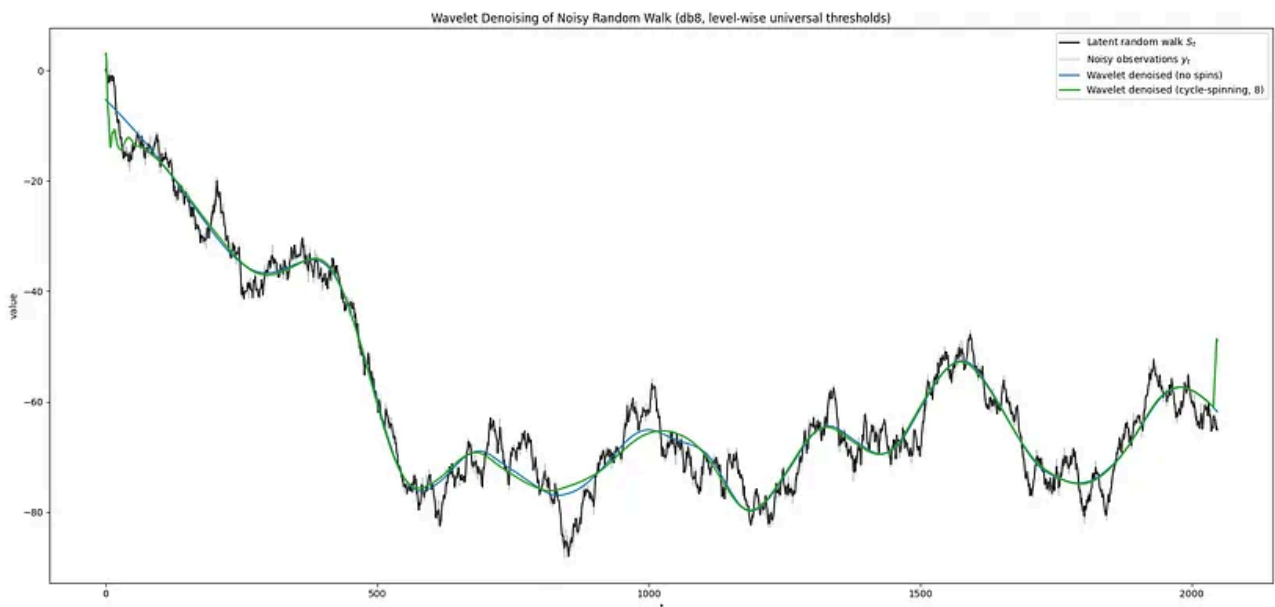
Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our [privacy policy](#).

```
plt.ylabel("value")
plt.legend()
plt.tight_layout()
plt.show()

# Print a quick report of thresholds (coarsest -> finest)
print("Detail thresholds (coarsest to finest):")
print([round(v, 3) for v in thr])
```

The following chart is the output of the code.



🌟 *The **Weekly Market Sentiment Report** is evolving into **The Signal Beyond** 🚀.*

This isn't just a sentiment check anymore. It's becoming a full market intelligence package

Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our [privacy policy](#).

Free trial available.



7 Likes

Discussion about this post

Comments

Restacks



Write a comment...

© 2026 Sofien Kaabar · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture

Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our [privacy policy](#).